

Chapitre 1

Le langage SQL

Rappel sur la sémantique des opérateurs

Ce que nous allons revoir

objectif "se remettre à niveau" avant la suite

1. Requêtes de base

projection, sélection, tri

2. Jointures

combinaison, dupliquer, conserver

3. Groupements et agrégations

changer la granularité du résultat

L'objectif

Même si vous (ou l'IAg) savez déjà *écrire* du SQL, il est indispensable de savoir **prédire** ce qu'une requête renvoie – sans l'exécuter – et de savoir **justifier** qu'elle répond bien à la question posée. C'est ce qu'on va retravailler !

Notre base d'exemples

un extrait de **mini-pagila**, le vidéoclub qui vous suivra tout le semestre

film

id	title	length	rating	rate	cat
1	Academy Dinosaur	86	PG	0.99	4
2	Bucket Brotherhood	133	PG	4.99	1
5	Escape Metropolis	141	R	4.99	1
6	Frost Head	78	PG	0.99	5
8	Hunting Musketeers	65	R	4.99	1
9	Island Empire	NULL	PG-13	1.99	3

category

category_id	name
1	Action
3	Documentaire
4	Animation
5	Horreur

customer

id	last_name	city
1	Dupont	Aix
2	Martin	Marseille
8	Petit	Toulon

rental

id	film	client	return_date
1	2	1	2025-01-08
2	5	1	2025-01-12
3	6	2	2025-01-11
16	9	2	NULL

return_date à NULL = location en cours.

Trois anomalies volontaires

elles vont nous servir toute la séance

Une durée inconnue

Island Empire n'a pas de length.

→ peut avoir un impact lors de comparaisons.

Des films jamais loués

Academy Dinosaur et
Hunting Musketeers n'apparaissent pas dans rental.

→ les jointures vont devoir décider de leur sort.

Un client sans location

Petit n'a rien loué.

→ qu'est-ce qu'on fait de lui?

À retenir

Une base « propre » n'existe généralement que dans les cours. Les cas limites (valeur absente, ligne orpheline, ensemble vide) sont exactement là où les requêtes fausses deviennent dangereuses, car les erreurs passent souvent inaperçues.

Partie 1

Anatomie d'une requête

ce qu'une requête fait vraiment, dans quel ordre

SQL est déclaratif

vous décrivez le résultat, pas la façon de l'obtenir

Impératif (ce que vous feriez en Java)

```
List<String> res = new ArrayList<>();
for (Film f : films) {
    if (f.getLength() < 90) {
        res.add(f.getTitle());
    }
}
Collections.sort(res);
```

Vous décrivez *comment* parcourir, tester, accumuler, trier.

Déclaratif (SQL)

```
SELECT title
FROM film
WHERE length < 90
ORDER BY title;
```

Vous décrivez *ce que vous voulez*. Le SGBD choisit le parcours, l'usage des index, l'algorithme de tri.

À retenir

Conséquence directe : il n'y a pas d'ordre des lignes sans **ORDER BY**, et pas de « première ligne trouvée ». Deux exécutions de la même requête peuvent renvoyer les mêmes lignes dans un ordre différent.

Les sept clauses

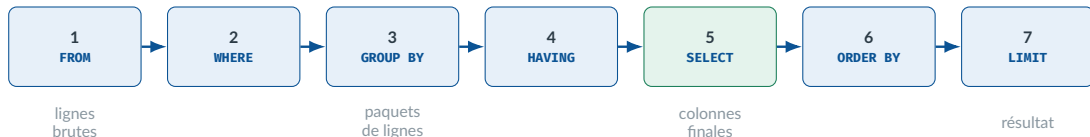
le squelette complet d'une requête

SELECT	quelles colonnes garder, et quelles expressions calculer
FROM	quelles tables lire, et comment les relier
WHERE	quelles lignes garder
GROUP BY	comment regrouper les lignes en "paquets" de lignes
HAVING	quels paquets garder
ORDER BY	dans quel ordre présenter le résultat
LIMIT	combien de lignes renvoyer

C'est l'ordre dans lequel on *écrit*. Ce n'est pas l'ordre dans lequel le SGBD *évalue*.

L'ordre d'évaluation logique

la diapo à connaître par cœur



Trois conséquences

- ▶ **WHERE** filtre *avant* le regroupement, **HAVING** *après* : un `COUNT()` ne peut pas apparaître dans **WHERE**.
- ▶ Un alias défini dans **SELECT** est utilisable dans **ORDER BY** (évalué après), pas dans **WHERE** (évalué avant).
- ▶ **LIMIT** s'applique en dernier, sur un résultat *déjà trié*.

```
SELECT rating, AVG(length) AS duree
FROM film
WHERE rental_rate > 1           -- lignes
GROUP BY rating                 -- paquets
HAVING COUNT(*) >= 2            -- paquets
ORDER BY duree DESC;           -- alias OK ici
```

Remplacer `HAVING COUNT(*)` par `WHERE COUNT(*)` : *aggregate functions are not allowed in WHERE.*

Partie 2

Requêtes de base

projection, sélection, tri

SELECT : la projection

on choisit des colonnes – le nombre de lignes ne change pas

film

title	length	rating	rate
Academy Dinosaur	86	PG	0.99
Bucket Brotherhood	133	PG	4.99
Escape Metropolis	141	R	4.99
Frost Head	78	PG	0.99
Hunting Musketeers	65	R	4.99
Island Empire	NULL	PG-13	1.99



```
SELECT title, length FROM film;
```

title	length
Academy Dinosaur	86
Bucket Brotherhood	133
Escape Metropolis	141
Frost Head	78
Hunting Musketeers	65
Island Empire	NULL

6 lignes en entrée, 6 lignes en sortie.

SELECT *

Pratique en exploration, à éviter en production : le résultat change si quelqu'un ajoute une colonne, et vous récupérez des données dont vous n'avez pas besoin / pas prévues à l'origine.

DISTINCT : supprimer les doublons

s'applique à la ligne *entière* du résultat, pas à une colonne

```
SELECT rating FROM film;
```

rating

PG
PG
R
PG
R
PG-13

6 lignes

```
SELECT DISTINCT rating  
FROM film;
```

rating

PG
R
PG-13

3 lignes, ordre non garanti

Attention

SELECT DISTINCT rating,
title ne dédouble pas
les rating : il dédouble
les *couples* (rating, title).

Remarque

DISTINCT n'est pas gratuit : il faut trier ou hacher tout le résultat. Mieux vaut comprendre *pourquoi* des doublons existent (et si c'est grave) avant de l'ajouter par réflexe.

WHERE : la sélection

on choisit des lignes – les colonnes ne changent pas

film

title	length	rating
Academy Dinosaur	86	PG
Bucket Brotherhood	133	PG
Escape Metropolis	141	R
Frost Head	78	PG
Hunting Musketeers	65	R
Island Empire	NULL	PG-13

Chaque ligne est testée indépendamment des autres.



```
SELECT title, length, rating
FROM film
WHERE length > 100;
```

title	length	rating
Bucket Brotherhood	133	PG
Escape Metropolis	141	R

À retenir

Island Empire est écarté, alors qu'on ne sait *pas* si sa durée dépasse 100. On en re-parle juste après.

La boîte à outils du WHERE

Condition	Exemple	Se lit
comparaison	<code>length >= 90</code>	la durée atteint 90 min
égalité de chaîne	<code>rating = 'R'</code>	<i>exactement</i> 'R' (sensible à la casse)
appartenance	<code>rating IN ('G','PG')</code>	raccourci d'un OR
intervalle	<code>length BETWEEN 80 AND 120</code>	bornes incluses
motif	<code>title LIKE 'B%'</code>	commence par B
absence de valeur	<code>length IS NULL</code>	la durée est inconnue
négation	<code>NOT (...), <>, NOT IN</code>	<i>*inverse la condition*</i>

À garder à l'esprit
Attention aux **NULL** !

Deux précisions sur le filtrage

Les motifs de LIKE

% = n'importe quelle suite de caractères (même vide).

→ Ex : '%o%' = contient un o.

_ = exactement un caractère.

→ Ex : '___' = fait 3 caractères.

ILIKE : version insensible à la casse (PostgreSQL)

Erreur classique

rating = 'PG' ne sélectionne **pas** les films 'PG-13' : ce sont deux chaînes différentes.

rating LIKE 'PG%', si.

NULL n'est pas une valeur

c'est l'absence d'information, et ça change la logique

Trois choses différentes :

- 0 une valeur : zéro
- " une valeur : chaîne vide
- NULL** on ne sait pas

La logique n'est plus à deux valeurs mais à trois :

expression	résultat
141 > 100	vrai
86 > 100	faux
NULL > 100	inconnu
NULL = NULL	inconnu
NULL <> 100	inconnu

La règle unique

WHERE ne garde une ligne que si la condition vaut **vrai**.

Inconnu est traité comme **faux** : la ligne est écartée.

Conséquence

`WHERE length > 100` et `WHERE length <= 100`
écartent **toutes les deux** Island Empire.

Pour tester l'absence (ou l'existence) :

```
WHERE length IS NULL           -- durée inconnue
WHERE length IS NOT NULL        -- durée connue
```

NULL : exemple sur les données

```
WHERE length <> 86
```

title	length
Academy Dinosaur	86
Bucket Brotherhood	133
Escape Metropolis	141
Frost Head	78
Hunting Musketeers	65
Island Empire	NULL

4 lignes. Island Empire disparaît sans qu'on l'ait demandé.

```
WHERE length < 80  
OR length IS NULL
```

title	length
Academy Dinosaur	86
Bucket Brotherhood	133
Escape Metropolis	141
Frost Head	78
Hunting Musketeers	65
Island Empire	NULL

3 lignes : « court ou durée inconnue ».

Le réflexe

Devant toute colonne pouvant valoir **NULL**, posez-vous la question :

« et si la valeur est absente, cette ligne doit-elle sortir ? »

Puis écrivez la réponse explicitement. Ne pas décider, c'est décider de l'exclure.

COALESCE(length, 0) permet de remplacer **NULL** par une valeur de remplacement – utile à l'affichage, dangereux dans un calcul.

ORDER BY : le tri

la seule façon d'avoir un ordre garanti

```
SELECT title, length FROM film
ORDER BY length DESC;
```

title	length
Island Empire	NULL
Escape Metropolis	141
Bucket Brotherhood	133
Academy Dinosaur	86
Frost Head	78
Hunting Musketeers	65

Surprise : Island Empire arrive en tête.

Spécificité PostgreSQL

En tri **décroissant**, les **NULL** sont placés en **premier**; en tri croissant, en dernier.

Un « top 3 des films les plus longs » écrit naïvement renvoie donc un film de durée inconnue en première place.

```
ORDER BY length DESC NULLS LAST --
      explicite
-- ou bien filtrer en amont :
WHERE length IS NOT NULL
```

Tri multiple : ORDER BY nb DESC, title – le second critère départage les ex æquo. Sans lui, l'ordre entre ex æquo n'est pas déterministe.

LIMIT : les N premiers

et le piège des ex æquo

« Les 2 films les plus chers »

```
SELECT title, rental_rate FROM film
ORDER BY rental_rate DESC
LIMIT 2;
```

title	rental_rate
Bucket Brotherhood	4.99
Escape Metropolis	4.99
Hunting Musketeers	4.99

Trois films sont à 4.99. **LIMIT** en coupe un arbitrairement – et le choix peut varier d'une exécution à l'autre.

Deux questions différentes

« Donne-moi 2 lignes » → **LIMIT** convient.

« Donne-moi les films *au tarif maximum* » → **LIMIT** est faux, il faut comparer à la valeur maximale :

```
SELECT title, rental_rate FROM film
WHERE rental_rate = (SELECT MAX(rental_rate)
                     FROM film);
```

→ les 3 films à 4.99.

PostgreSQL propose aussi `FETCH FIRST 2 ROWS WITH TIES`.

Partie 3

Jointures

combiner des tables sans perdre ni inventer de lignes

Pourquoi joindre ?

l'information est répartie, par construction

La table `film` ne contient pas le *nom* de la catégorie, seulement son identifiant :

title	category_id
Frost Head	5
Island Empire	3

C'est voulu : le nom « Horreur » n'est stocké **qu'une fois**, dans `category`. Le renommer se fait en une seule écriture, et aucun film ne peut référencer une catégorie inexistante.

À retenir

La jointure est le prix à payer pour recoller une information volontairement répartie entre plusieurs tables.

```
SELECT f.title, c.name
FROM   film f
       JOIN category c
       ON f.category_id = c.category_id;
```

Les alias (`f`, `c`) ne sont pas décoratifs : dès que deux tables ont une colonne de même nom, ils deviennent obligatoires pour lever l'ambiguïté.

Étape 1 : le produit cartésien

toutes les combinaisons possibles

film (3 lignes)

title	cat
Bucket Brotherhood	1
Frost Head	5
Island Empire	3

category (2 lignes)

id	name
1	Action
5	Horreur

$3 \times 2 = 6$ combinaisons.



```
SELECT * FROM film, category;      -- produit cartésien
```

title	cat	id	name
Bucket Brotherhood	1	1	Action
Bucket Brotherhood	1	5	Horreur
Frost Head	5	1	Action
Frost Head	5	5	Horreur
Island Empire	3	1	Action
Island Empire	3	5	Horreur

La jointure interne, c'est ce produit **plus un filtre** : ne garder que les lignes où $cat = id$ (en vert).

JOIN : la jointure interne

seules les lignes qui trouvent un partenaire restent

```
SELECT f.title, c.name
FROM   film f
       JOIN category c ON f.category_id = c.
                        category_id;
```

title	name
Academy Dinosaur	Animation
Bucket Brotherhood	Action
Escape Metropolis	Action
Frost Head	Horreur
Hunting Musketeers	Action
Island Empire	Documentaire

À retenir

JOIN = INNER JOIN. « Interne » signifie : une ligne sans partenaire **disparaît** du résultat.

Deux écritures équivalentes ?

FROM film f JOIN category c ON ...

✓ recommandé

FROM film f, category c WHERE ...

✗ ancien style

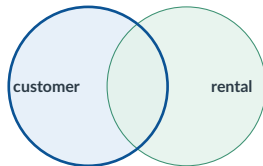
La seconde mélange condition de jointure et filtre métier. Un **WHERE** oublié transforme silencieusement la requête en produit cartésien.

LEFT JOIN / RIGHT JOIN : conserver les lignes sans partenaire

```
SELECT cu.last_name, r.rental_id
FROM   customer cu
       LEFT JOIN rental r
       ON cu.customer_id = r.customer_id;
```

last_name	rental_id
Dupont	1
Dupont	2
Martin	3
Martin	16
Petit	NULL

Petit est conservé. Les colonnes de `rental` sont remplies de **NULL**.



LEFT JOIN = tout le disque de gauche, complété quand c'est possible

À retenir

LEFT JOIN garde **toutes** les lignes de la table de gauche. **RIGHT JOIN** fait l'inverse (on peut toujours s'en passer en échangeant les tables). **FULL JOIN** garde les deux côtés.

Le motif « anti-jointure »

comment exprimer « ceux qui n'ont pas de ... »

Les films jamais loués

```
SELECT f.title
FROM   film f
       LEFT JOIN rental r ON f.film_id = r.film_id
WHERE  r.rental_id IS NULL;
```

title	rental_id
Bucket Brotherhood	1
Escape Metropolis	2
Frost Head	3
Island Empire	16
Academy Dinosaur	NULL
Hunting Musketeers	NULL

1. le **LEFT JOIN** conserve tout le monde 2. le **WHERE** ne garde que les lignes de remplissage.

Le test doit porter sur une colonne **NOT NULL**

On teste `r.rental_id` (la clé primaire), jamais `r.return_date` : cette dernière vaut **NULL** aussi bien pour « pas de location » que pour « location en cours ». On ne saurait plus distinguer les deux.

Deux autres écritures du même besoin :

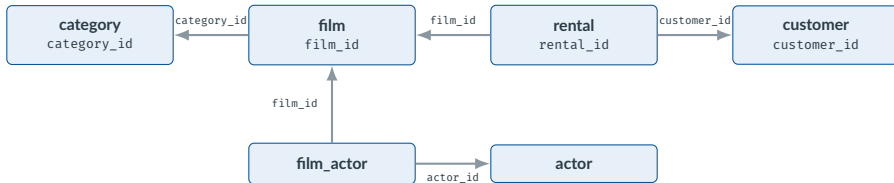
```
WHERE f.film_id NOT IN (SELECT film_id FROM
                        rental)
```

```
WHERE NOT EXISTS (SELECT * FROM rental r
                  WHERE r.film_id = f.
                        film_id)
```

La dernière est la plus sûre : voir la diapo sur **NOT IN** et les **NULL**.

Chaîner les jointures

suivre le chemin des clés étrangères



« Les clients ayant loué un film d'action »

```
SELECT DISTINCT cu.last_name
FROM   customer cu
      JOIN rental r   ON cu.customer_id = r.
customer_id
      JOIN film f     ON r.film_id      = f.film_id
      JOIN category c ON f.category_id = c.
category_id
WHERE  c.name = 'Action';
```

À retenir

On ne joint pas « les tables dont on a besoin » : on suit le **chemin** qui les relie. Ici, deux des quatre tables ne fournissent aucune colonne au résultat. Ce sont des passerelles.

Partie 4

Groupements et agrégations

changer la granularité du résultat



GROUP BY : des lignes aux "paquets" de lignes

film

title	length	rating
Academy Dinosaur	86	PG
Bucket Brotherhood	133	PG
Frost Head	78	PG
Escape Metropolis	141	R
Hunting Musketeers	65	R
Island Empire	NULL	PG-13

1. le SGBD range les lignes en paquets de même

rating



```
SELECT rating, COUNT(*) AS nb, AVG(length) AS moy
FROM film
GROUP BY rating;
```

rating	nb	moy
PG	3	99.0
R	2	103.0
PG-13	1	NULL

2. chaque paquet est réduit à une seule ligne, par les fonctions d'agrégation.

La règle du SELECT après regroupement

✗ Refusé

```
SELECT  rating, title, COUNT(*)
FROM    film
GROUP BY rating;
```

column "film.title" must appear in the GROUP BY clause or be used in an aggregate function

✓ Accepté

```
SELECT  rating, COUNT(*), MIN(title)
FROM    film
GROUP BY rating;
```

MIN(title) = une valeur unique par groupe.

La règle

Toute colonne du **SELECT** doit être :

- ▶ soit présente dans le **GROUP BY**,
- ▶ soit passée à une fonction d'agrégation.

Ce n'est pas une contrainte arbitraire : c'est la seule façon d'avoir *une* valeur par groupe.

Les fonctions d'agrégation et les **NULL**

la différence entre « combien de lignes » et « combien de valeurs »

Sur le groupe PG-13, qui contient la seule ligne
Island Empire (durée **NULL**) :

expression	résultat
COUNT(*)	1
COUNT(length)	0
AVG(length)	NULL
SUM(length)	NULL
MAX(length)	NULL

Sur la table entière :

expression	résultat
COUNT(*)	6
COUNT(length)	5
AVG(length)	100.6
COUNT(DISTINCT rating)	3

Deux règles, et tout en découle

1. COUNT(*) compte les **lignes** du groupe.
Toutes les autres fonctions – y compris
COUNT(colonne) – **ignorent les NULL**.
2. Sur un ensemble vide, COUNT renvoie **0** ;
toutes les autres renvoient **NULL**.

Piège

Dans un groupe contenant une durée inconnue,
la moyenne est calculée **sur les valeurs connues
seulement** : on divise par COUNT(length),

Le piège des groupes vides

nombre de locations par personne (exemple)

✗ Jointure interne

```
SELECT cu.last_name, COUNT(*)
FROM   customer cu JOIN rental
      r
      ON cu.customer_id = r.
      customer_id
GROUP BY cu.customer_id, cu.
      last_name;
```

last_name	count
Dupont	2
Martin	2

Petit disparaît : aucune ligne, donc aucun groupe.

✗ LEFT JOIN + COUNT(*)

```
SELECT cu.last_name, COUNT(*)
FROM   customer cu LEFT JOIN
      rental r
      ON cu.customer_id = r.
      customer_id
GROUP BY cu.customer_id, cu.
      last_name;
```

last_name	count
Dupont	2
Martin	2
Petit	1

Petit revient, mais à 1 : sa ligne de remplissage compte.

✓ LEFT JOIN + COUNT(col)

```
SELECT cu.last_name, COUNT(r.
      rental_id)
FROM   customer cu LEFT JOIN
      rental r
      ON cu.customer_id = r.
      customer_id
GROUP BY cu.customer_id, cu.
      last_name;
```

last_name	count
Dupont	2
Martin	2
Petit	0

COUNT(colonne) ignore le NULL de remplissage.

« Y compris ceux qui n'en ont aucun » ⇒ jointure externe et COUNT(colonne).

WHERE OU HAVING ?

filtrer des lignes, ou filtrer des groupes



Les deux ensemble, et ce n'est pas redondant

```
SELECT rating, COUNT(*) AS nb
FROM film
WHERE rental_rate > 1      -- j'écarte des films
GROUP BY rating
HAVING COUNT(*) >= 2;      -- j'écarte des
                           groupes
```

Sur notre extrait : le **WHERE** écarte les 2 films à 0.99; restent 4 films, soit les groupes R (2), PG (1) et PG-13 (1); le **HAVING** ne garde que R. Résultat : R | 2.

À retenir

Un agrégat dans **WHERE** est **toujours** une erreur : au moment où **WHERE** s'exécute, les groupes n'existent pas encore.

Règle d'optimisation

Ce qui peut être filtré en **WHERE** doit l'être : on écarte les lignes avant de regrouper.

Partie 5

Sous-requêtes

quand une requête a besoin du résultat d'une autre

Trois usages, trois formes

1. Scalaire : elle renvoie une valeur

```
SELECT title FROM film
WHERE rental_rate > (SELECT AVG(rental_rate)
                     FROM film);
```

Évaluée une seule fois. Utilisable partout où une valeur est attendue.

2. Ensemble : elle renvoie une colonne

```
SELECT title FROM film
WHERE category_id IN (SELECT category_id
                     FROM category
                     WHERE name = 'Action');
```

IN, NOT IN, ANY, ALL. La sous-requête doit renvoyer une seule colonne.

3. Corrélée : elle dépend de la ligne courante

```
SELECT c.last_name FROM customer c
WHERE NOT EXISTS (SELECT * FROM rental r
                  WHERE r.customer_id = c.
                    customer_id);
```

L'alias *c* est mentionné à l'intérieur : la sous-requête est ré-évaluée pour chaque client. EXISTS s'arrête à la première ligne trouvée.

À retenir

Sous-requête **non corrélée** : calculable seule, une fois pour toutes.

Sous-requête **corrélée** : dépend de la ligne examinée. Utilise un ou plusieurs alias de la requête externe.

NOT IN et les NULL

le piège le plus coûteux du chapitre

Supposons qu'une location ait un `customer_id` à **NULL** :

```
SELECT last_name FROM customer
WHERE customer_id NOT IN (SELECT customer_id
                          FROM rental);
```

Pour le client 8, SQL évalue :

`8 <> 1 AND 8 <> 2 AND 8 <> NULL`

= **vrai** AND **vrai** AND **inconnu** = **inconnu**

⇒ ligne écartée. **Le résultat est vide**, quelle que soit la donnée.

Piège

Dès qu'une seule valeur **NULL** apparaît dans la sous-requête, `NOT IN` renvoie un résultat vide, silencieusement. Et c'est indétectable en lisant la requête : il faut connaître la nullabilité de la colonne.

Réflexe

Préférer `NOT EXISTS`, qui n'a pas ce comportement :

```
WHERE NOT EXISTS (SELECT * FROM rental r
                  WHERE r.customer_id = c.
                  customer_id)
```

`IN` n'a pas ce problème : c'est bien la *négation* (`NOT IN`) qui le crée.

Partie 6

Pour conclure

bravo, vous avez survécu (jusqu'ici)

Six questions à se poser

avant de rédiger une requête (liste non exhaustive)

1. Une ligne de mon résultat, c'est quoi ?

Un film ? une location ? un groupe ? La réponse doit correspondre à l'énoncé.

2. Où se situent les informations dont j'ai besoin ?

Dans quelles tables ? Comment celles-ci sont-elles reliées ?

3. Filtrer avant ou après le regroupement ?

WHERE pour les lignes, **HAVING** pour les groupes.

4. En cas de jointure, les lignes « sans rien » doivent-elles apparaître ?

Si oui : jointure externe, et potentiellement `COUNT(colonne)` plutôt que `COUNT(*)`.

5. Que se passe-t-il si la valeur est absente ?

NULL n'est ni vrai ni faux. La ligne sort-elle ou non ?

6. Ai-je des doublons, et sont-ils voulus ?

DISTINCT n'est pas automatique, c'est un choix.

À retenir

Ces six questions ne se répondent pas en exécutant la requête. Elles se répondent en la lisant ou en l'écrivant.

En TD (sur feuille)

- ▶ traduire une requête en français,
- ▶ juger si un résultat est cohérent,
- ▶ compléter, corriger, écrire,
- ▶ reconstruire une requête à partir de son résultat,
- ▶ expertiser une requête produite par une IA.

Pas de machine : c'est volontaire. On ne peut pas « essayer jusqu'à ce que ça marche », il faut raisonner.

En TP (sur machine)

- ▶ la base mini-pagila complète sous PostgreSQL,
- ▶ vos requêtes, exécutées et vérifiées,
- ▶ les messages d'erreur, qu'il faudra savoir lire.

À retenir

Le TD entraîne le raisonnement, le TP entraîne la mise en œuvre.